Lab 2 – Smart City

Team Gold

Old Dominion University

CS411W

Dr. Sumaya Sanober

2026 Version

Table of Contents

1 Introduction 3

1.1 Purpose 3

1.2 Scope 3

1.3 Definitions, Acronyms, and Abbreviations 3

1.4 References 4

1.5 Overview 4

2 Overall Description 5

2.1 Product Perspective 5

2.2 Product Functions 6

2.3 User Characteristics 7

2.4 Constraints 7

2.5 Assumptions and Dependencies 7

List of Figures

Figure 1:Smart City Major Functional Component Diagram 5

List of Tables

Table 1:Table of Comparison Between RWP and Prototype 6
Table 2:Cont'd Table of Comparison Between RWP and Prototype 7

1 Introduction

Cities are getting bigger and more crowded every year. The United Nations says that by 2050, around 6.5 billion people will be living in cities. Because of this, people deal with more traffic, less space, and slower communication between city systems. The way most cities are built can't handle this kind of growth, so things like traffic, pollution, and poor resource management keep getting worse.

Smart City is our solution to this problem. It's a community based app that helps people move around the city faster and stay informed. The app makes smarter route suggestions and shares useful updates. The main goal is to reduce congestion and make it easier for residents, commuters, and city officials to stay connected. Smart City helps people and technology work together so cities can be more organized, efficient, and enjoyable to live in.

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional scope, system context, user roles, constraints, and dependencies for the Smart City prototype. The document serves as a technical reference for developers and project stakeholders involved in the design implementation, and evaluation of the system. The purpose of this document is to provide a clear description of the system's functionality and structure so that developers can understand how the system should behave during implementation.

1.2 Scope

Smart City is an application designed to provide location based navigation and community interaction features for users within a city environment. The system integrates mapping functionality with a community forum to allow users to discover locations, share posts tied to specific map locations, and interact with other users.

The prototype will demonstrate the core functionality of the system, including displaying map data, creating and viewing forum posts, and interacting with posts tied to map locations. The prototype focuses on demonstrating the integration between the map interface and the community forum system.

1.3 Definitions, Acronyms, and Abbreviations

API (Application Programming Interface): A set of protocols that allows different software systems to communicate with each other.

Backend: The portion of the system responsible for processing logic, handling requests, and managing data storage.

Database: A structured system used to store and manage application data.

Frontend: The user-facing portion of the application responsible for displaying information and receiving user input.

Forum Post: A user-generated message associated with a specific location on the map that can be viewed and interacted with by other users.

Map API: A third-party service, Mapbox, used to provide geographic map data and location services.

MFCD (Major Functional Component Diagram): A diagram illustrating the main system components and how they interact.

1.4 References

GitHub. (2025, December 3). Where the world builds software. GitHub. https://github.com/

React Native. (2025, December 3). React Native. React Native. https://reactnative.dev/

Expo. (2025, December 3). The fastest way to make an app. Expo. https://expo.dev/

Joyce, H. (2025, March 26). Urban overcrowding: A major driver of pollution crises. ShunWaste. https://shunwaste.com/article/how-is-overcrowding-in-cities-causing-more-pollution

Lee, S. (2025, May 27). Understanding urban overcrowding: The social and economic implications of overcrowding in cities. Number Analytics. https://www.numberanalytics.com/blog/ultimate-guide-to-overcrowding-in-urban-sociology

Weinberg, N. (2016, August 11). How can cities be preemptive and effective in preventing overcrowding? Data-Smart City Solutions. https://datasmart.hks.harvard.edu/news/article/how-can-cities-be-preemptive-and-effective-in-preventing-overcrowding-888

Luoma, K. (2016, November 21). How does population growth impact rapid urbanization? Population Education. https://populationeducation.org/how-does-population-growth-impact-rapid-urbanization/

1.5 Overview

Section 2 of this document provides the overall description of the Smart City prototype, including the product perspective, major product functions, user characteristics, constraints, and assumptions and dependencies.

2 Overall Description

2.1 Product Perspective

The Smart City prototype is a mobile application that integrates location based navigation with a community discussion platform. The system allows users to view a map of their surrounding area and interact with posts that are associated with specific locations.

The system consists of several major components including the mobile application interface, backend services, and a database used for storing user data and forum posts. The mobile frontend allows users to interact with the map interface, create posts, and view community discussions. The backend handles application logic, processes requests from the mobile client, and manages communication with the database.

The system also interacts with external services to support key functionality. A third-party mapping API, Mapbox, is used to provide map visualization and location services. These services allow the system to display geographic data and associate forum posts with specific coordinates on the map.

The architecture of the Smart City prototype consists of a presentation layer, application layer, and data layer. The relationships between these components are illustrated in the Major Functional Component Diagram (MFCD).

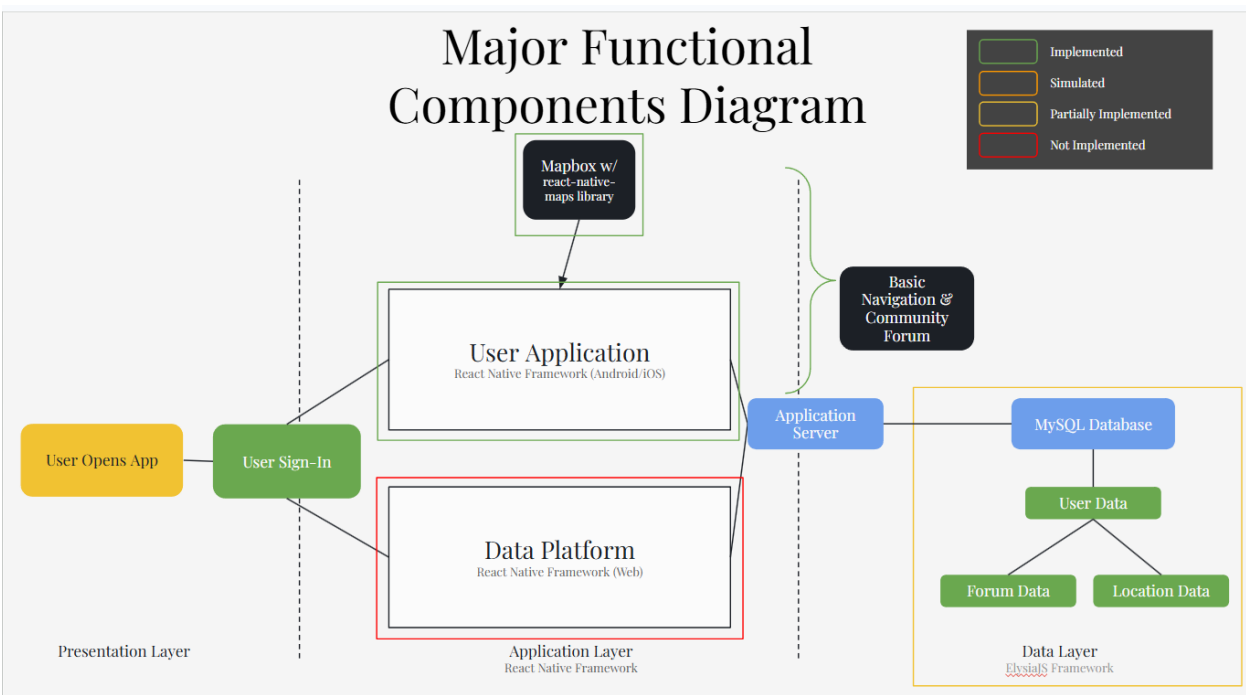

Figure 1:Smart City Major Functional Component Diagram

2.2 Product Functions

The Smart City prototype includes features related to account management, navigation, community interaction, and data handling. These features represent a reduced version of the Real World Product (RWP) and were selected based on what could reasonably be completed during the project timeline.

Account management features such as login/authentication, account creation and deletion, and managing account permissions are fully implemented because they are necessary for users to access the system. Users can also access the community forum, which is fully implemented in the prototype.

Some features are only partially implemented. The dynamic queue system for navigation was simplified to basic navigation due to complexity and time constraints. Alerts and notifications are currently simulated until full API support can be implemented.

Several features were removed from the prototype scope. On the data platform side, the central information hub is partially implemented, while data analytics and data visualization were removed because they depended on a long-term data platform that is no longer part of the prototype. Data anonymity remains fully implemented to ensure user privacy.

Table 1:Table of Comparison Between RWP and Prototype

Category	Features	RWP	Current Prototype	Reasoning
Account Management	Login/Authentication	Fully Implemented	Fully Implemented	Essential for prototype
	Account Creation/Deletion	Fully Implemented	Fully Implemented	Essential for prototype
	Manage Account Access/Permissions	Fully Implemented	Fully Implemented	Essential for prototype
User Application	Access Community Forum	Fully Implemented	Fully Implemented	Essential for prototype
	Access Dynamic Queue System for Navigation	Fully Implemented	Partially Implemented	We removed the Dynamic Queue implementation as it would be too complex for the given time frame. Still implementing the basic navigation.
	Alerts and Notifications	Fully Implemented	Partially Implemented	Simulated feature until we figure out how to handle API calls.
	Add Parking locations	Fully Implemented	Not Implemented	No time given scope of project. Could reconsider

Table 2:Cont'd Table of Comparison Between RWP and Prototype

Category	Features	RWP	Current Prototype	Reasoning
Data Platform	Central Information Hub	Fully Implemented	Partially Implemented	Still will implement the database for storing information in. Will drop long-term data platform due to time constraints.
	Data Analytics and Insights	Fully Implemented	Not Implemented	Dropped due to no longer implementing the long-term data platform
	Visualize Data	Fully Implemented	Not Implemented	Dropped due to no longer implementing the long-term data platform
	Data Anonymity	Fully Implemented	Fully Implemented	Essential for prototype

2.3 User Characteristics

Smart City is designed for users with varying levels of technical experience. The primary users of the system are general mobile application users who wish to explore locations and interact with community posts related to those locations.

Most users are expected to have basic familiarity with smartphone applications and common user interface interactions such as navigation, scrolling, and selecting items on a map. No specialized technical knowledge is required to use the system.

2.4 Constraints

The Smart City prototype is subject to several technical constraints. The system must operate within the development environment and tools selected by the project team. The prototype is also limited by the available development time and resources for the project.

Additionally, the system relies on third-party services such as mapping APIs to provide geographic data. If these services are unavailable or limited, some system functionality may be affected.

2.5 Assumptions and Dependencies

The Smart City prototype assumes that users will have access to a mobile device with internet connectivity to access the application. The system also assumes that third-party mapping services will remain available to provide location data and map visualization.

The system depends on backend services, database infrastructure, and external APIs to support its core functionality. These components must function correctly for the system to operate as intended.